In-Class Case Study & Architecture Design Activity

University Course Registration System

Duration: 60–90 minutes | Group activity | Deliverable: one architecture diagram + justification

Learning Outcomes

- Identify architectural problems from a realistic software scenario.
- Explain why a system needs an explicit architecture rather than an ad-hoc structure.
- Identify components, responsibilities, and communication paths.
- Apply concepts such as REST, caching, asynchronous messaging, concurrency control, idempotency, server-side validation, scalability, and single points of failure.
- Justify architectural decisions using system requirements and quality attributes.

Case Study

A university wants to develop a Course Registration System for approximately 10,000 students. Students should be able to log in, view courses, register for courses, drop courses, and view their registrations. Administrators should be able to manage course information.

The university has the following requirements:

- A course has a maximum capacity. For example, Database Systems can have at most 100 students.
- The system must never allow the course capacity to be exceeded.
- A student must not be registered for the same course more than once.
- Only authenticated and authorised students may register.
- Registration data must be stored reliably.
- Students should receive a confirmation email after successful registration.
- Course information is read very frequently but changes relatively infrequently.
- During registration week, several thousand students may access the system simultaneously.

Your Starting Point: Assume There Is No Planned Architecture

Imagine that a development team starts coding immediately. They initially create the following simple structure:

Browser → One Application → One Database

The application contains login, registration, validation, email sending, reporting, and database access. No clear responsibilities or communication rules have been agreed.

Do not redesign the system yet. First, investigate whether this design can satisfy the requirements.

Step 1 – Identify the Problems

Discuss the following questions as a group. Record your answers.

1. If two students try to take the last available seat at exactly the same time, what could go wrong?

Group notes: _____

_____

2. If a student changes the order/registration amount using browser Developer Tools, should the server trust the value sent by the browser? Why?

Group notes: _____

_____

3. If a student double-clicks Register, what could happen?

Group notes: _____

_____

4. If thousands of students repeatedly view the course catalogue, what could happen to the database?

Group notes: _____

_____

5. If the system waits for the confirmation email before responding, what will the student experience?

Group notes: _____

_____

6. If the only database fails, what happens to the whole system?

Group notes: _____

_____

Step 2 – Match Each Problem to a Concept

Complete the table before designing your improved architecture.

Scenario	What could go wrong?	Relevant concept	Possible solution
Two students take the last seat		Concurrency / race condition	
Client changes a value in Developer Tools		Server-side validation	
Student double-clicks Register		Idempotency / duplicate prevention	
Thousands of course reads		Caching / performance	
Email does not need to delay the response		Asynchronous messaging	
Only database fails		Single point of failure / reliability	

Step 3 – Build the Improved Architecture

Now redesign the system. Do not start by choosing technologies. Start by identifying responsibilities.

Your architecture should show, where appropriate:

- Client/user interface
- REST API or application interface
- Business logic / registration rules
- Data access and database
- Caching for suitable read-heavy data
- A message queue and email service for asynchronous confirmation
- Authentication/authorisation boundary
- Mechanism for safe concurrent registration
- Mechanism to prevent duplicate registration
- A suitable approach to application availability/scalability

Draw your architecture below. Use boxes for major components and arrows for communication.

Step 4 – Label Responsibilities

For each major component in your diagram, state its main responsibility.

Component	Main responsibility	What it should NOT do

Step 5 – Design the REST API

Propose REST endpoints for the following operations. Use resources as nouns and appropriate HTTP methods.

1. Retrieve all courses.

Endpoint: _____ Method: _____

2. Retrieve Course 42.

Endpoint: _____ Method: _____

3. Create a new course.

Endpoint: _____ Method: _____

4. Update Course 42.

Endpoint: _____ Method: _____

5. Delete Course 42.

Endpoint: _____ Method: _____

6. Retrieve the registrations of Student 10.

Endpoint: _____ Method: _____

7. Register Student 10 for Course 42.

Endpoint: _____ Method: _____

Challenge: Which endpoint could be affected by a double-click/retry? What mechanism would you use to prevent duplicate registration?

Answer: _____

_____

Step 6 – Explain the Critical Registration Scenario

There is exactly ONE seat left in Course 42. Student A and Student B send registration requests at almost the same time.

Draw or describe what could happen in a race condition:

_____

_____

_____

Now describe how your architecture prevents the problem. Mention the database transaction/concurrency mechanism you would rely on.

_____

_____

_____

Step 7 – Justify Your Architectural Decisions

For each decision, complete the chain:

Requirement → Problem/Risk → Quality Attribute → Architectural Decision → Mechanism

Requirement	Problem/Risk	Quality Attribute	Architectural Decision	Mechanism

Step 8 – Challenge Your Architecture

Your lecturer will introduce each change. Update your architecture only if necessary.

1. Traffic increases from hundreds to several thousand simultaneous users.

Does the architecture need to change? Why? _____

_____

2. The course catalogue becomes even more read-heavy.

Does the architecture need to change? Why? _____

_____

3. The email service becomes temporarily unavailable.

Does the architecture need to change? Why? _____

_____

4. The university launches a mobile application.

Does the architecture need to change? Why? _____

_____

5. A second campus starts using the same system.

Does the architecture need to change? Why? _____

_____

Final Group Check

□ We identified the major components.

□ Each component has a clear responsibility.

□ The client does not directly access the database.

□ Business rules are enforced on the server.

□ Concurrent registration cannot exceed course capacity.

□ Duplicate registration is prevented.

□ Course reads can be handled efficiently.

□ Email processing does not unnecessarily delay registration.

□ We considered availability and single points of failure.

□ We can explain why each major architectural decision was made.

Group Presentation – 3 Minutes

Present your architecture using this structure:

1. Show the architecture diagram.

2. Explain the responsibility of each major component.

3. Explain how you handle the last-seat race condition.

4. Explain how you prevent duplicate registration.

5. Explain your use of caching and asynchronous messaging.

6. Identify one remaining risk or trade-off in your design.

Key Takeaway

Architecture is not simply a diagram or a collection of technologies.

Architecture connects system requirements and quality attributes to decisions about structure, responsibilities, communication, data, and failure handling.